

Categories for the Working Hacker

Philip Wadler
University of Edinburgh & IOHK
Lambda Days
22 February 2018

Smart Contracts

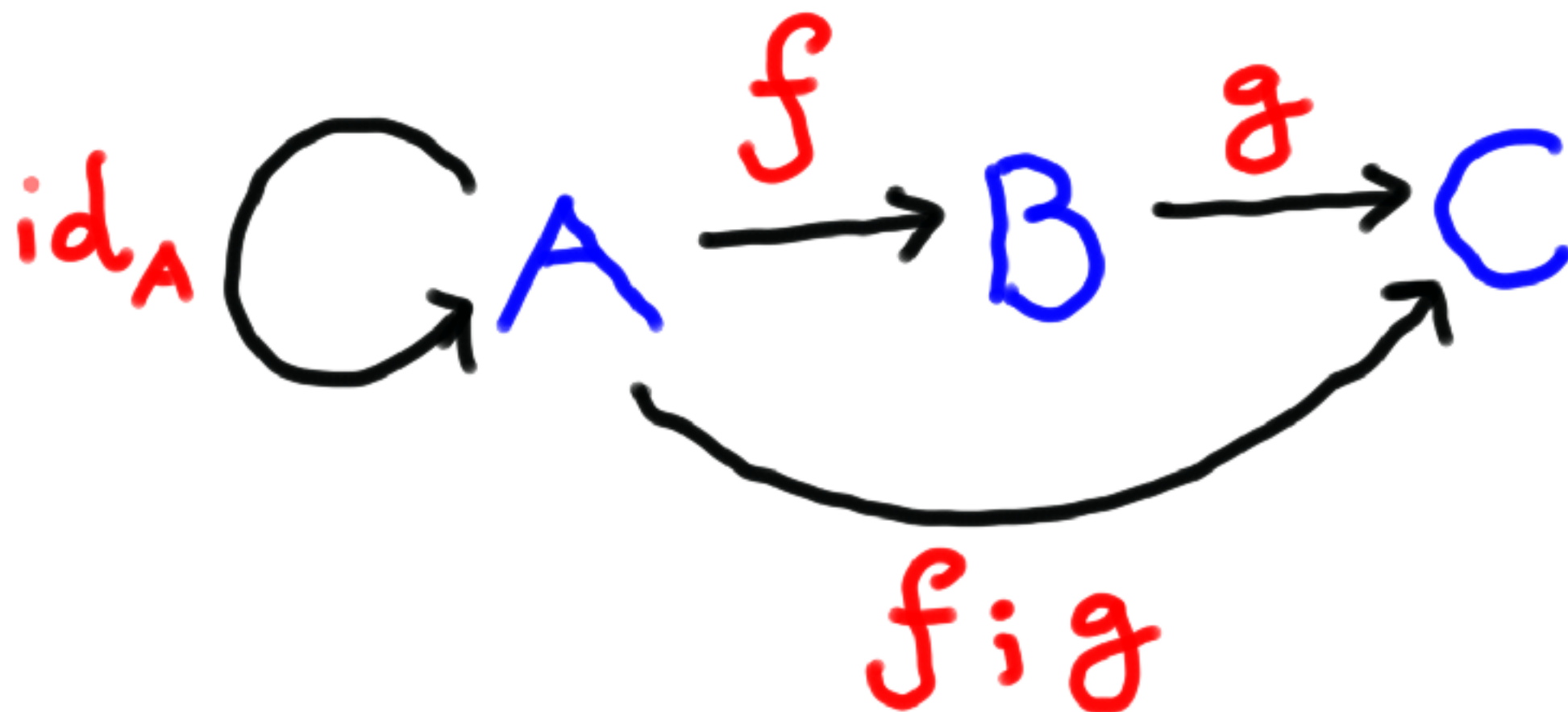
Simplicity

Michelson

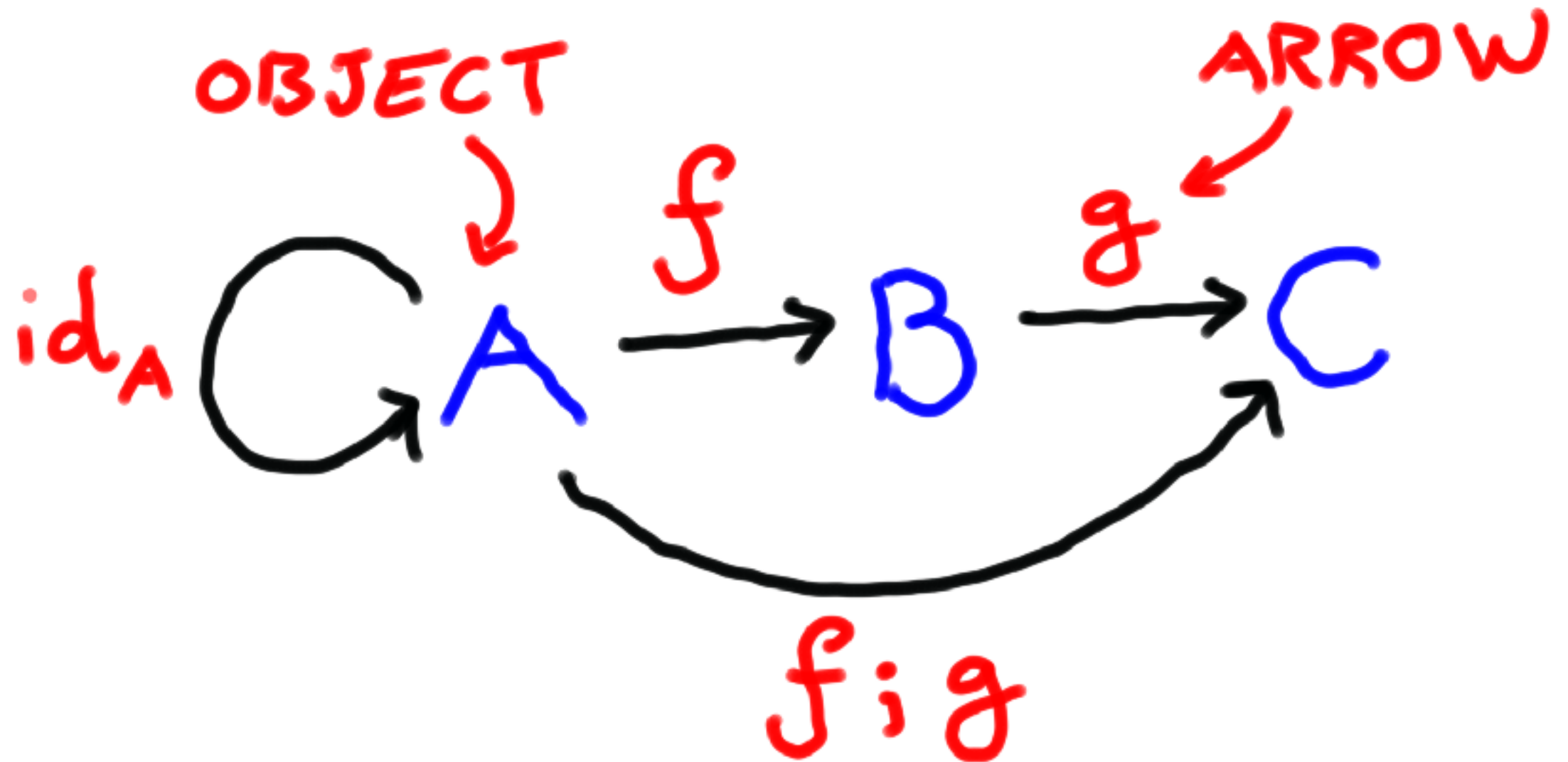
Plutus

Categories

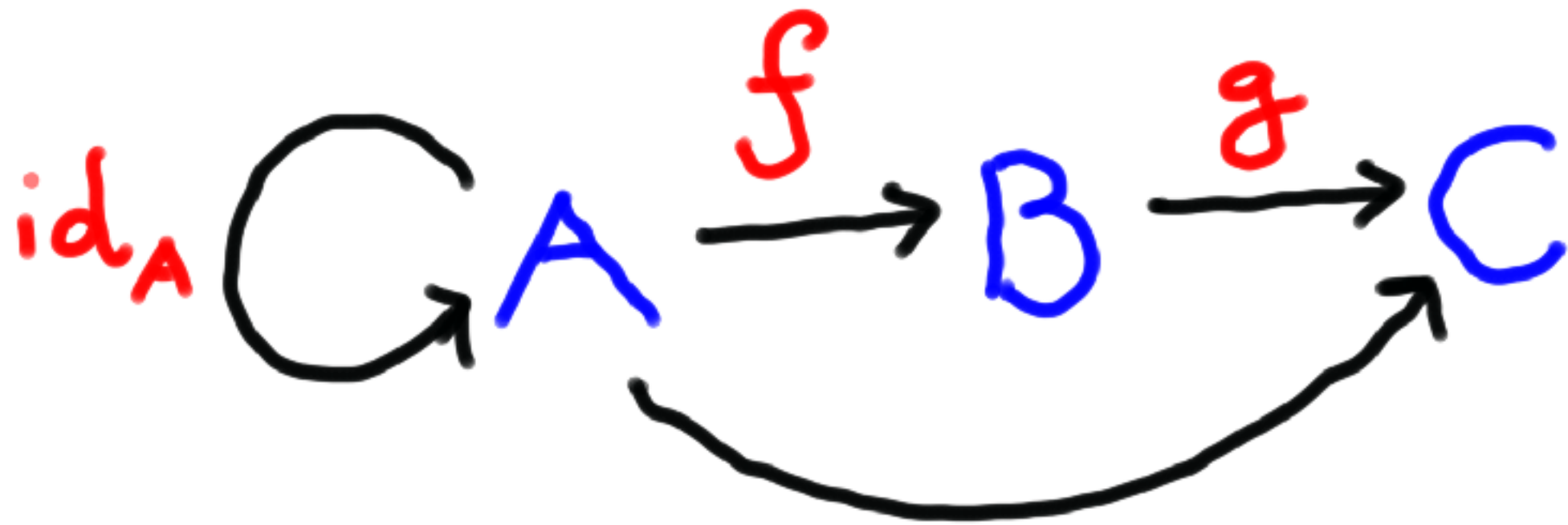
CATEGORIES



CATEGORIES



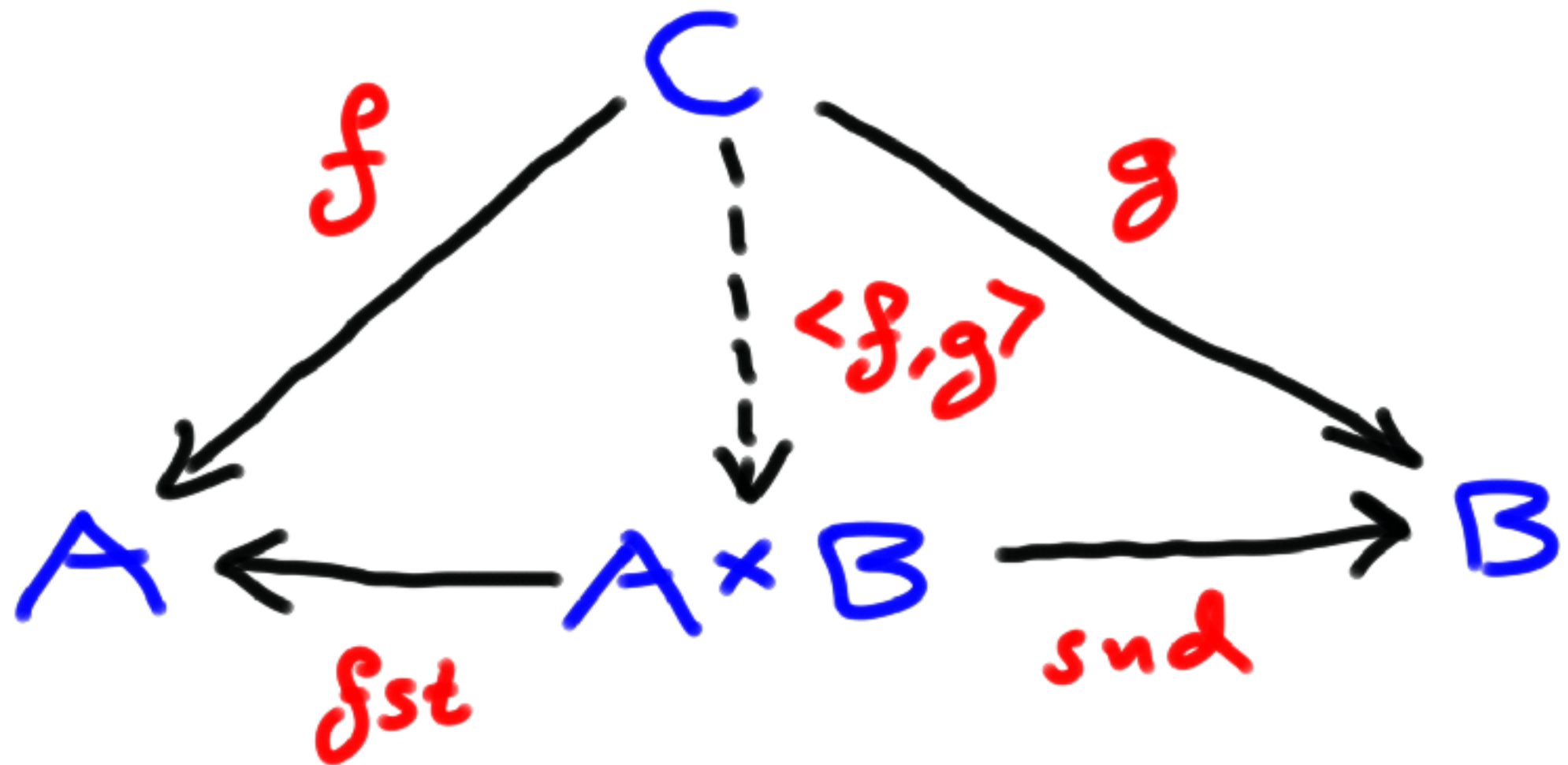
CATEGORIES



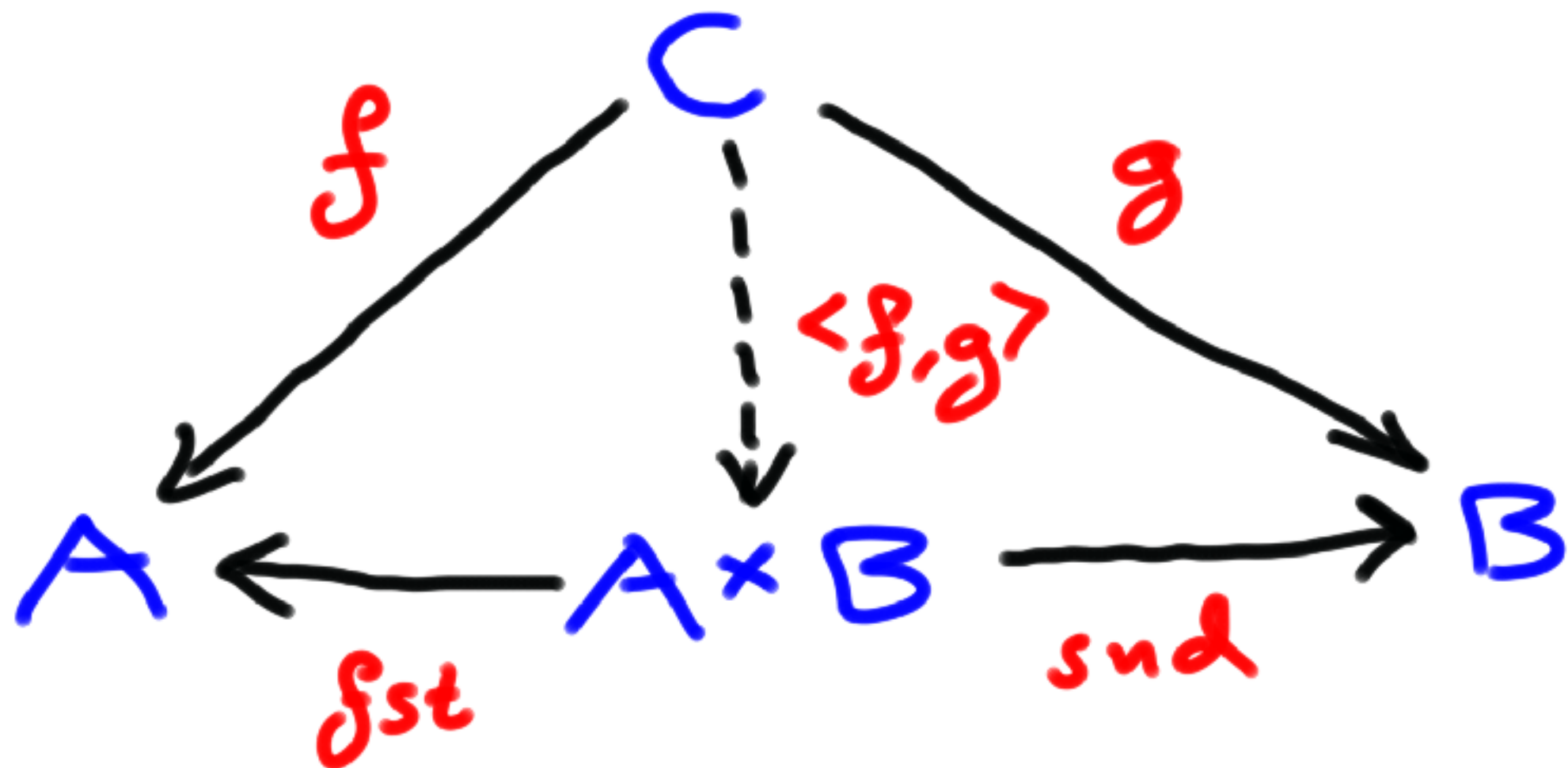
$$\begin{aligned} \text{id}_A \circ f &= f = f \circ \text{id}_B \\ (f \circ g) \circ h &= f \circ (g \circ h) \end{aligned}$$

Products

PRODUCTS

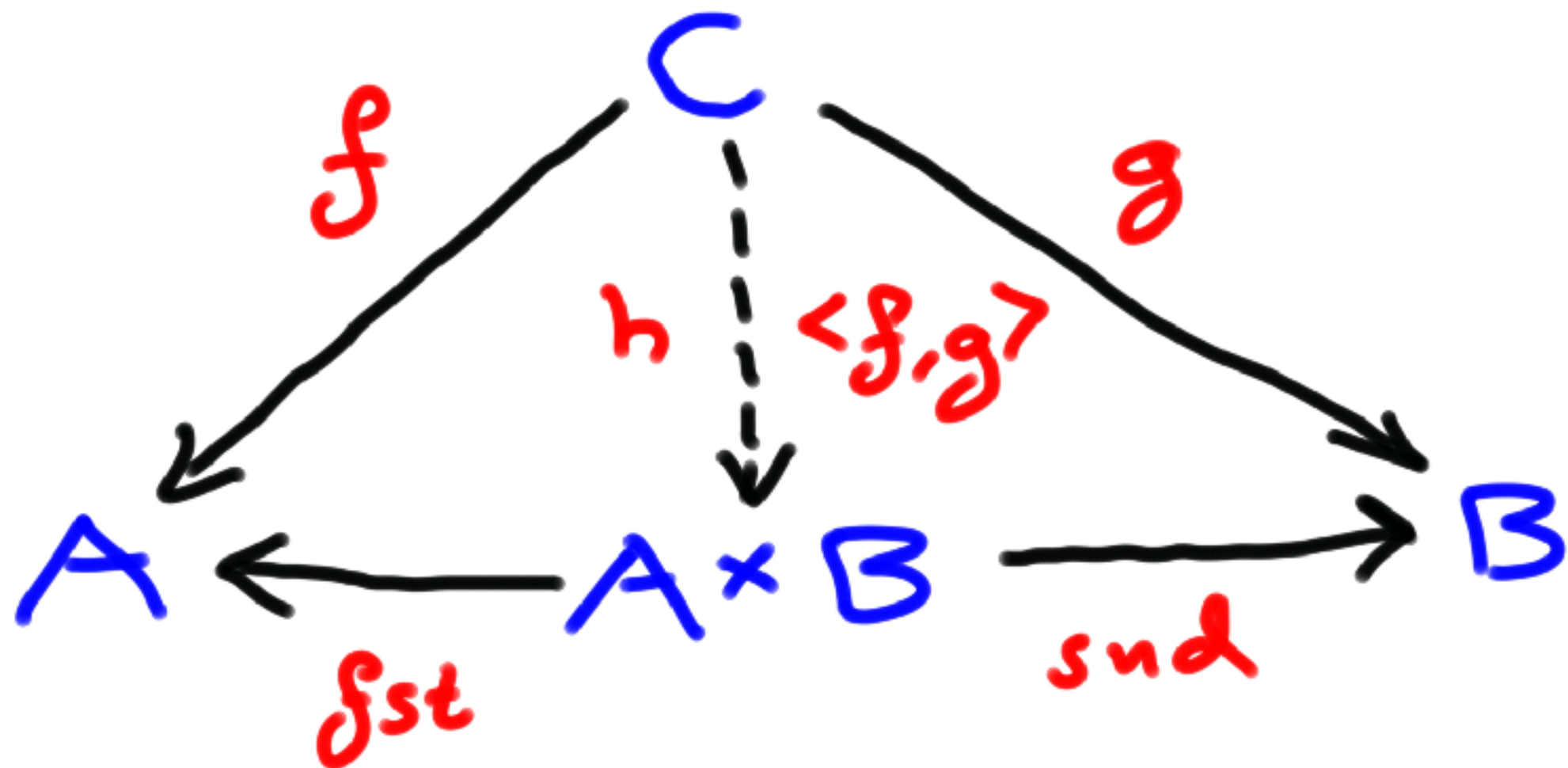


PRODUCTS



$$\begin{aligned}\langle f, g \rangle; f_{st} &= f \\ \langle f, g \rangle; snd &= g\end{aligned}$$

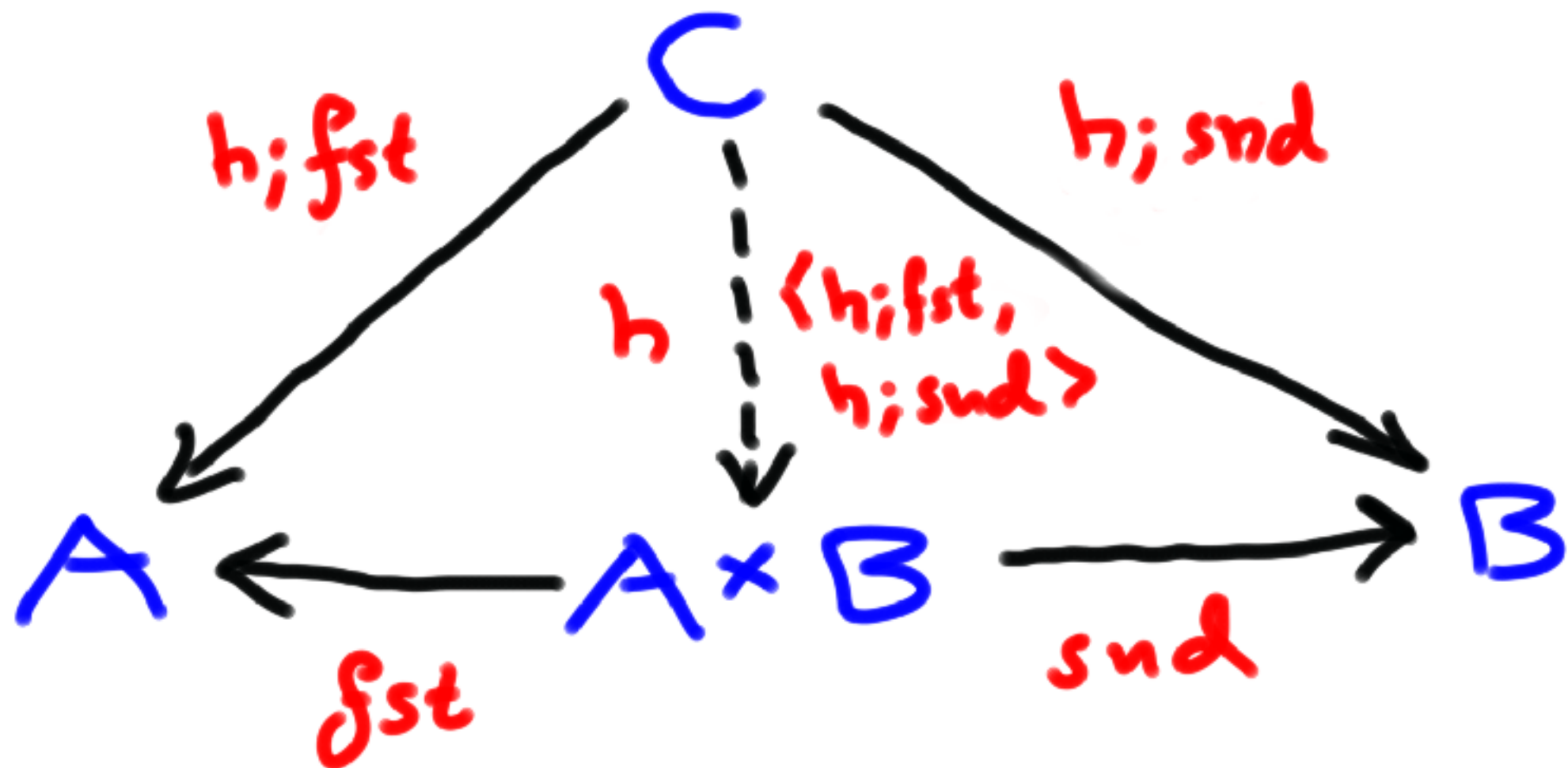
PRODUCTS



$$h; f_{st} = f \qquad h; snd = g$$

$$h = \langle f, g \rangle$$

PRODUCTS



$$h;fst = h;fst$$

$$h;snd = h;snd$$

$$h = \langle h;fst, h;snd \rangle$$

PRODUCTS

$$3 \times 2$$

$(a', 0)$

$(a', 1)$

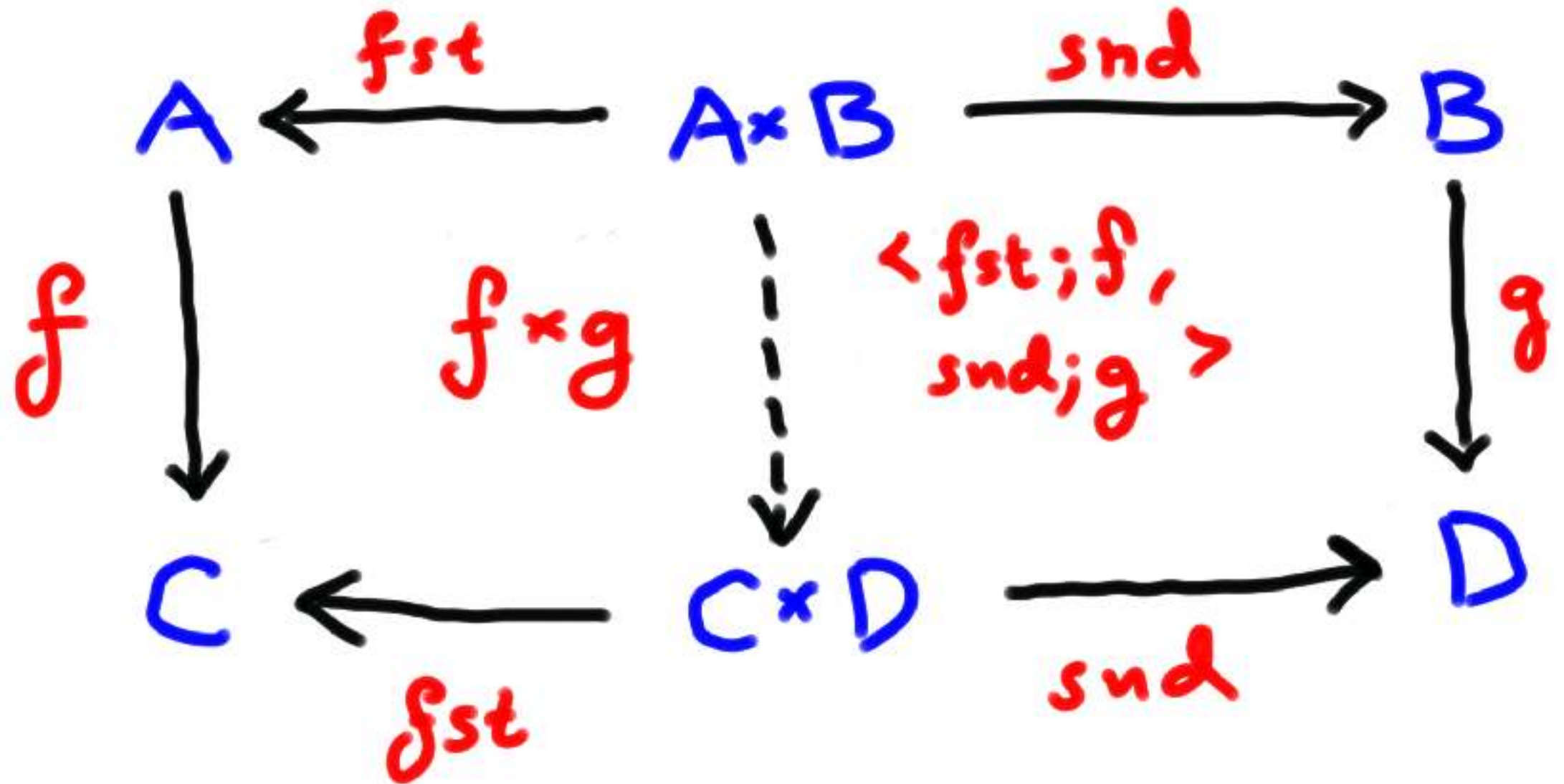
$(b', 0)$

$(b', 1)$

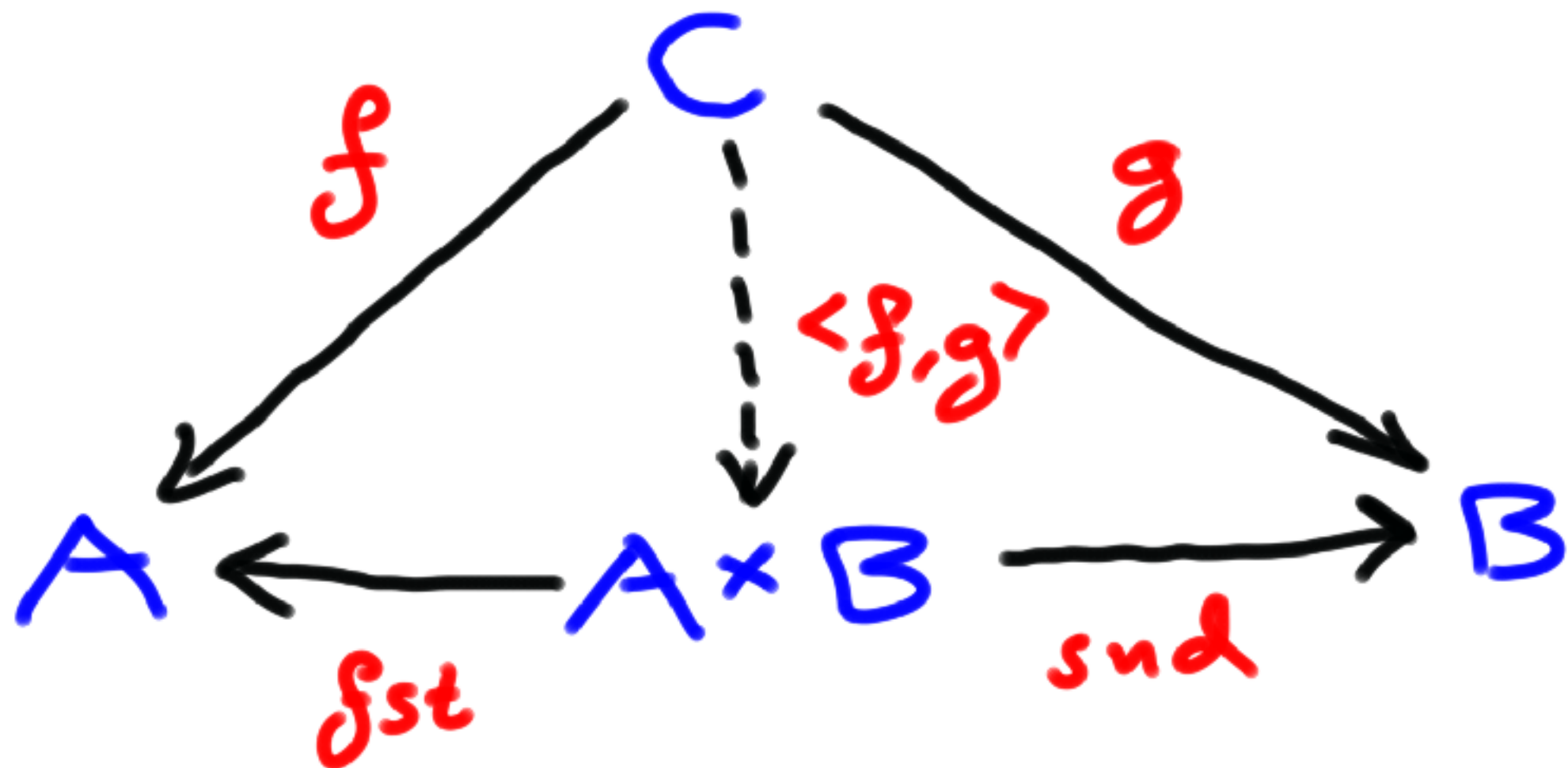
$(c', 0)$

$(c', 1)$

PRODUCTS



PRODUCTS



$$\mathcal{C}(C, A \times B) \cong \mathcal{C}(C, A) \times \mathcal{C}(C, B)$$

Products in Java

```
public class Product<A,B> {  
    private A fst;  
    private B snd;  
    public Product(A fst, B snd) {  
        this.fst = fst; this.snd = snd;  
    }  
    public A getFst() {  
        return this.fst;  
    }  
    public B getSnd() {  
        return this.snd;  
    }  
}
```


Products in Java

```
public class Test {  
    public Product<Integer,String> pair =  
        new Product(1, "two");  
    public Integer one = pair.getFst();  
    public String two = pair.getSnd();  
}
```


Products in Haskell

```
data Product a b =  
    Pair { fst :: a, snd :: b }
```

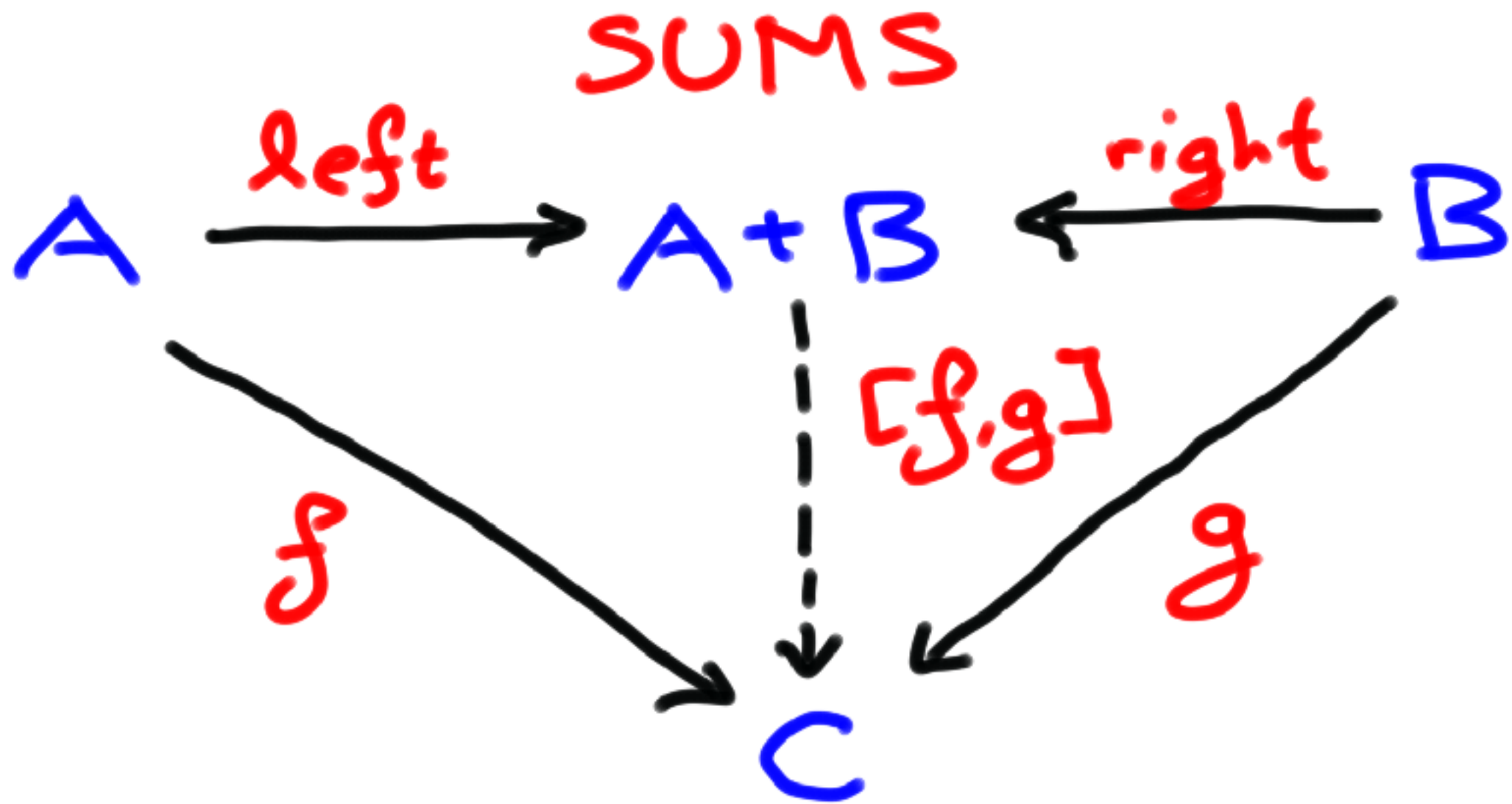
Products in Haskell

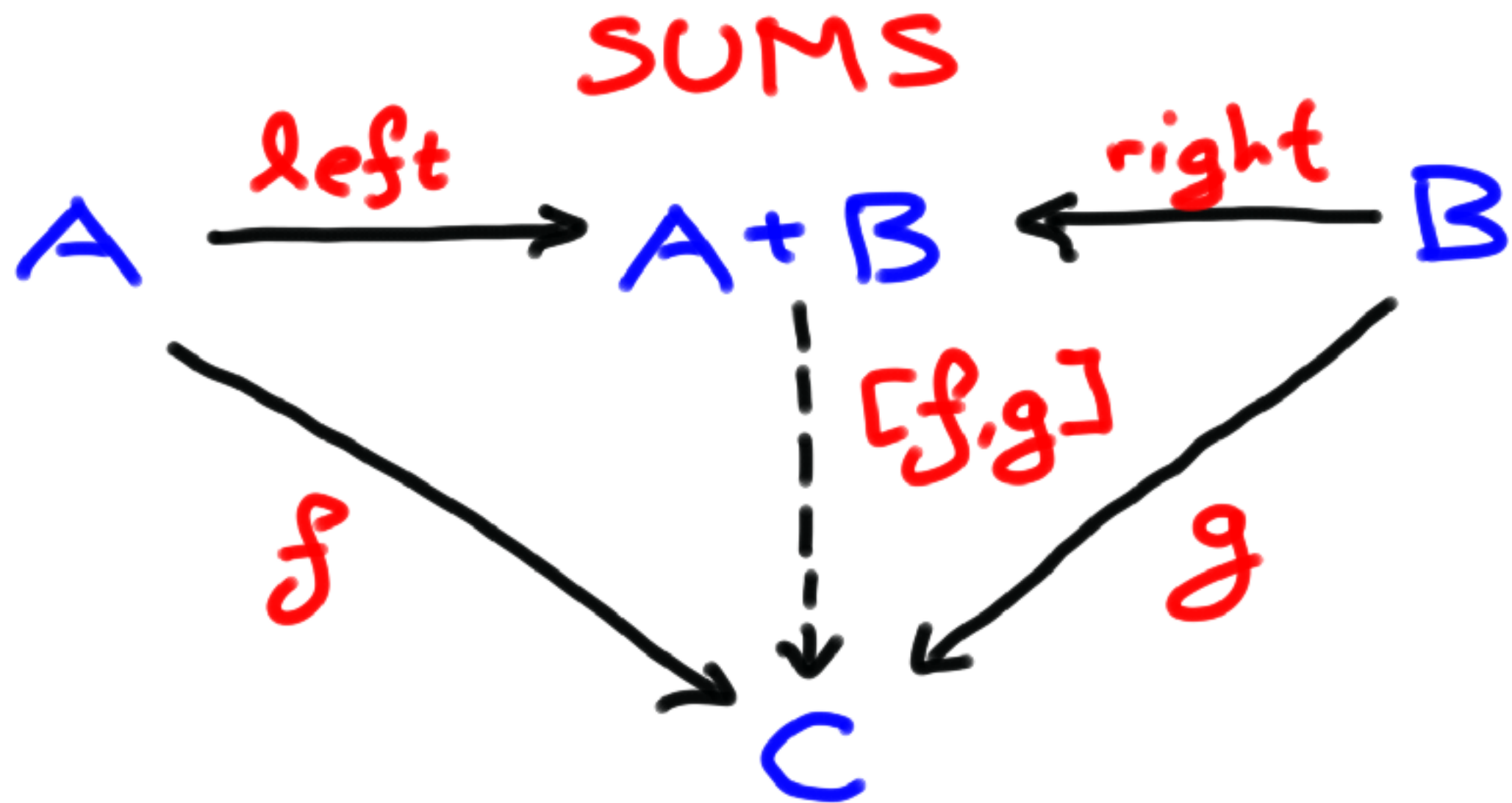
```
pair :: Product Int String  
pair = Pair 1 "two"
```

```
one :: Int  
one = fst pair
```

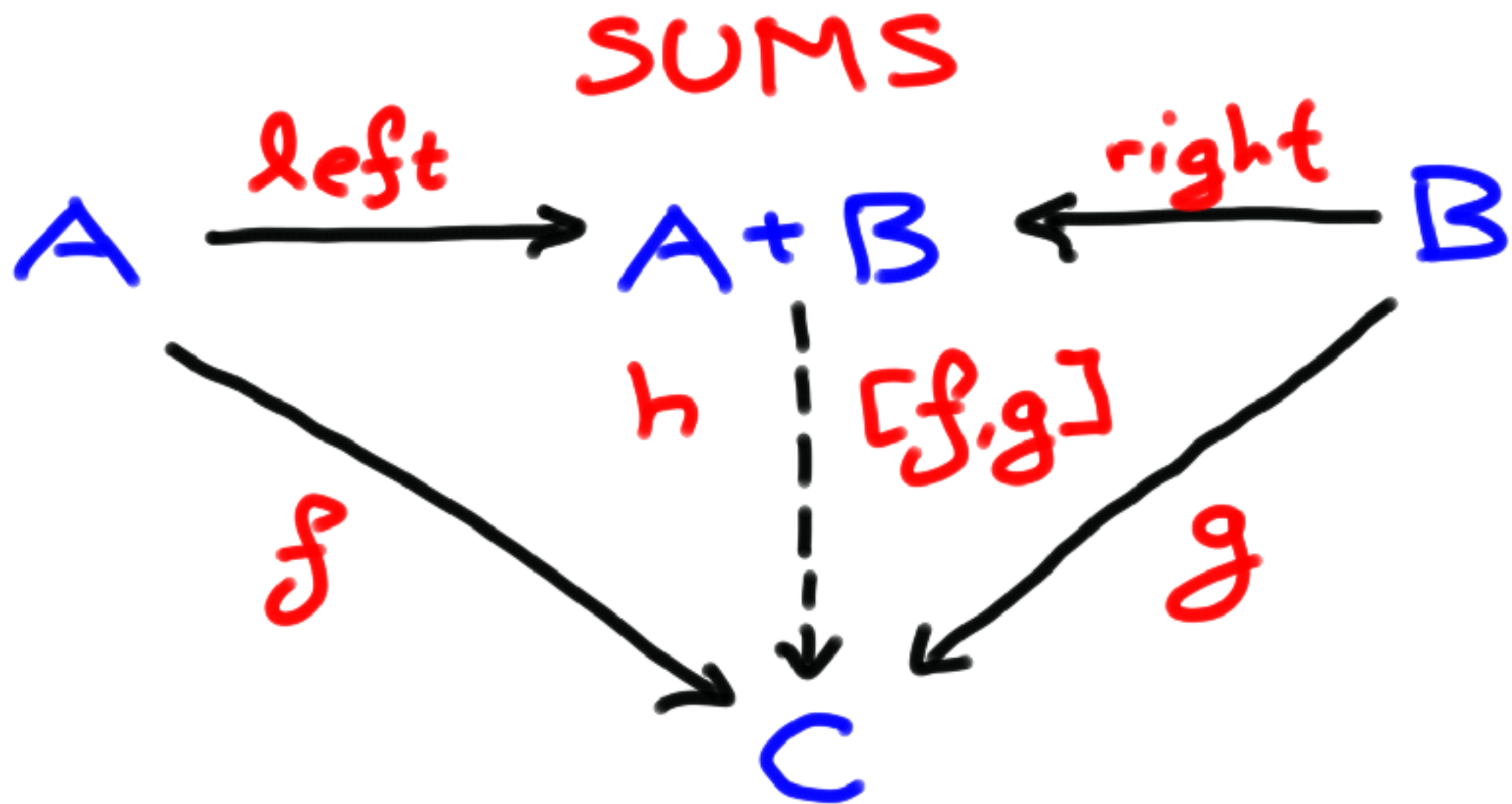
```
two :: String  
two = snd pair
```

Sums





$\text{left}; [f, g] = f$
 $\text{right}; [f, g] = g$



left; $h = f$ right; $h = g$
 $h = [f, g]$

Diagram illustrating the merge step of Merge Sort:

- Node **A** (left) and Node **B** (right) are merged into Node **A+B** (center).
- Node **A** is connected to **A+B** by a solid arrow labeled **left**.
- Node **B** is connected to **A+B** by a solid arrow labeled **right**.
- Node **A** is connected to Node **C** (bottom) by a solid arrow labeled **left; h**.
- Node **B** is connected to Node **C** by a solid arrow labeled **right; h**.
- Node **A+B** is connected to Node **C** by a dashed arrow labeled **h**.
- The text **[left; h, right; h]** is written in the center, indicating the recursive calls for the left and right subtrees.

left; h = left; h right; h = right; h
h = [left; h, right; h]

SUMS

$$3+2$$

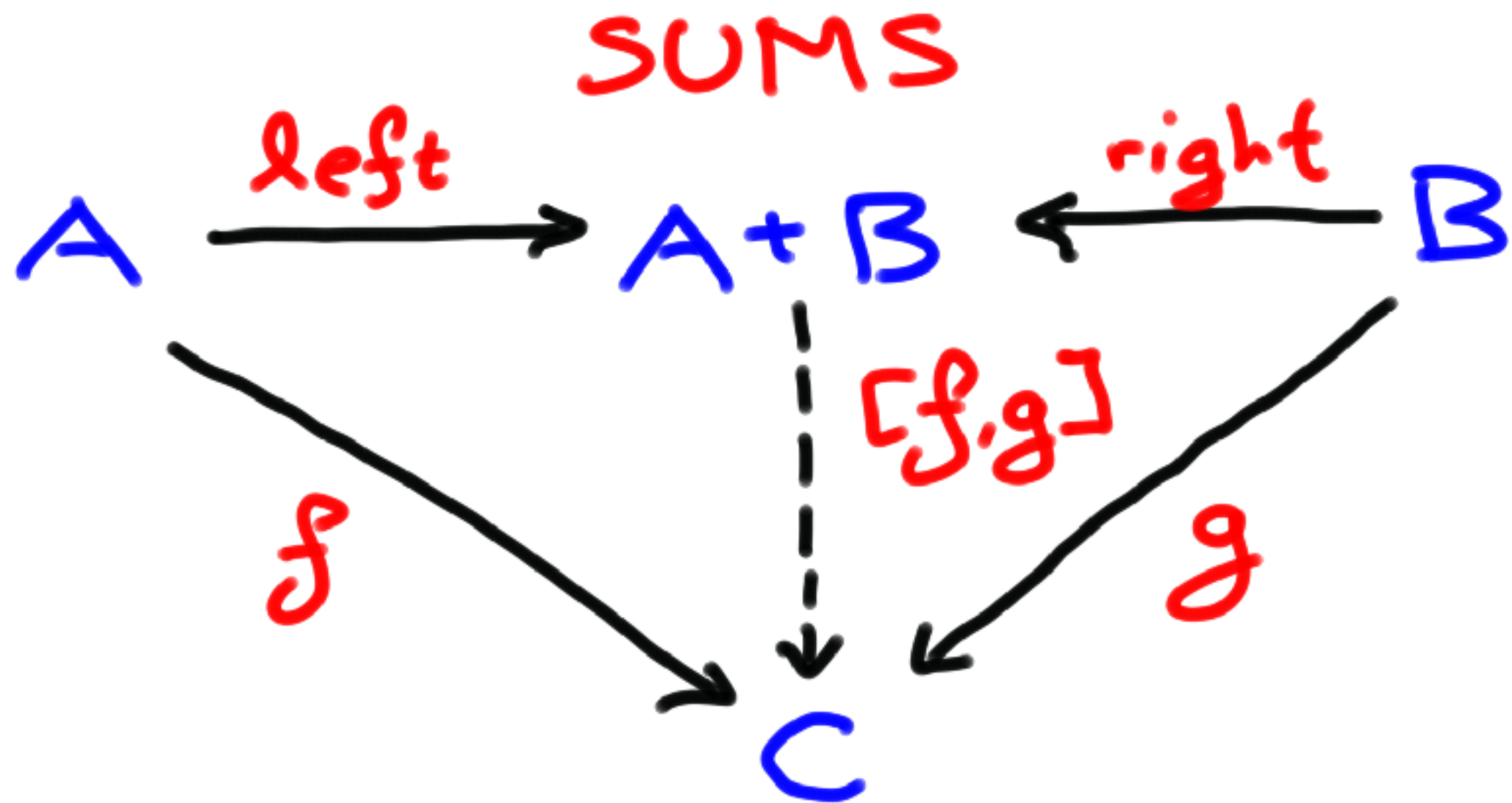
left 'a'

right 0

left 'b'

right 1

left 'c'



$$\mathcal{C}(A+B, C) \cong \mathcal{C}(A, C) \times \mathcal{C}(B, C)$$

Sums in Java

```
public interface Sum<A,B> {
    public <C> C caseExpr(Function<A,C> f,
                          Function<B,C> g);
}

public class Left<A,B> implements Sum<A,B> {
    private A x;
    public Left(A x) { this.x = x; }
    public <C> C caseExpr(Function<A,C> f,
                          Function<B,C> g) {
        return f.apply(x);
    }
}

public class Right<A,B> implements Sum<A,B> {
    private B y;
    public Right(B y) { this.y = y; }
    public <C> C caseExpr(Function<A,C> f,
                          Function<B,C> g) {
        return g.apply(y);
    }
}
```

Sums in Java

```
public class ErrInt extends Sum<String,Integer> {
    public ErrInt err = new Left("error");
    public ErrInt one = new Right(1);
    public ErrInt add(ErrInt that) {
        return this.caseExpr(
            e -> new Left(e),
            m -> that.caseExpr(
                e -> new Left(e),
                n -> new Right(m+n)
            )
        );
    }
    public ErrInt test = one.add(err);
}
```

Sums in Haskell

```
data Sum a b = Left a | Right b
```

Sums in Haskell

```
type ErrInt = Sum String Int
```

```
err = Left "error"
```

```
one = Right 1
```

```
add :: ErrInt -> ErrInt -> ErrInt
```

```
add (Left e)    that      = Left e
```

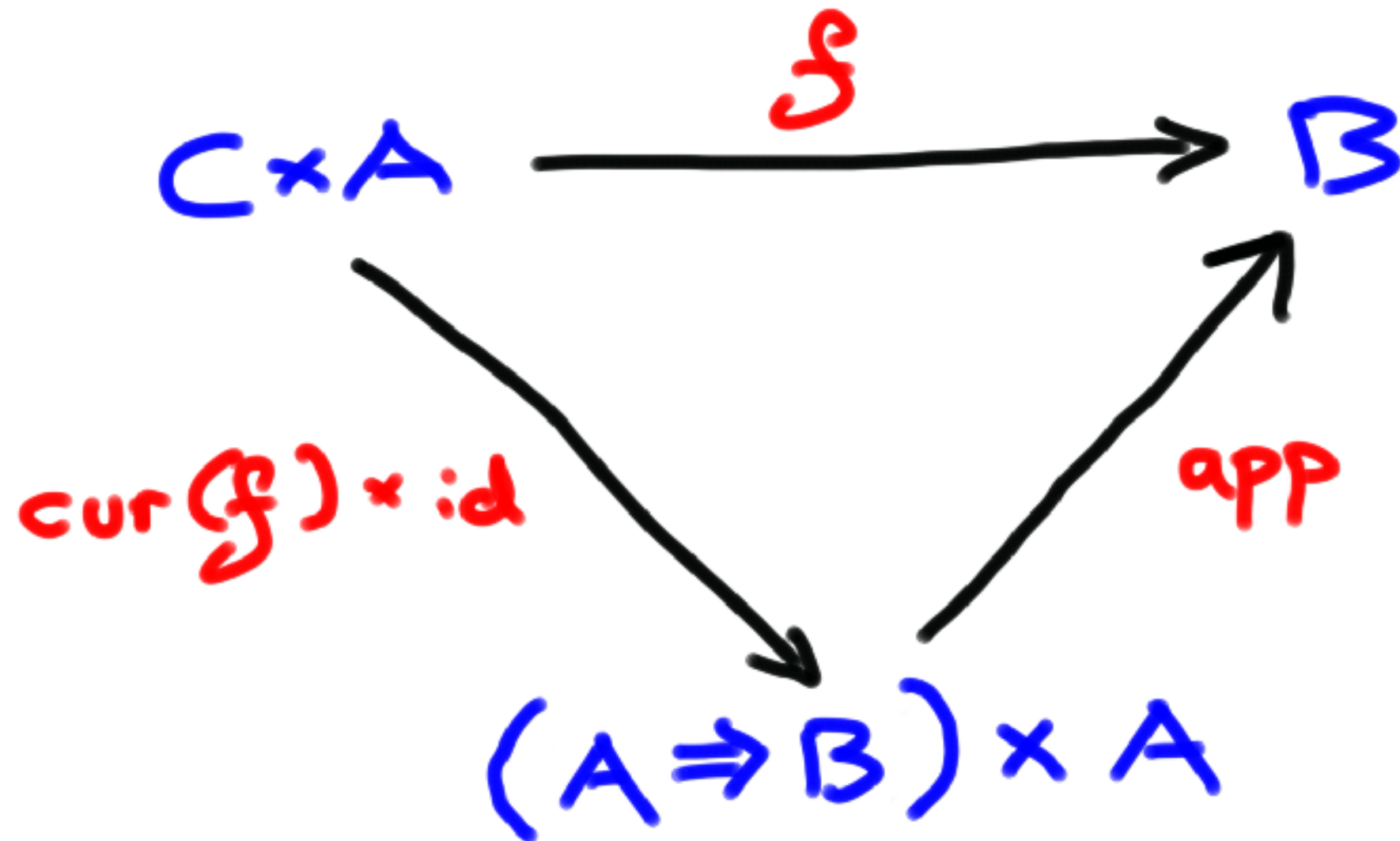
```
add this       (Left e)   = Left e
```

```
add (Right m) (Right n) = Right (m+n)
```

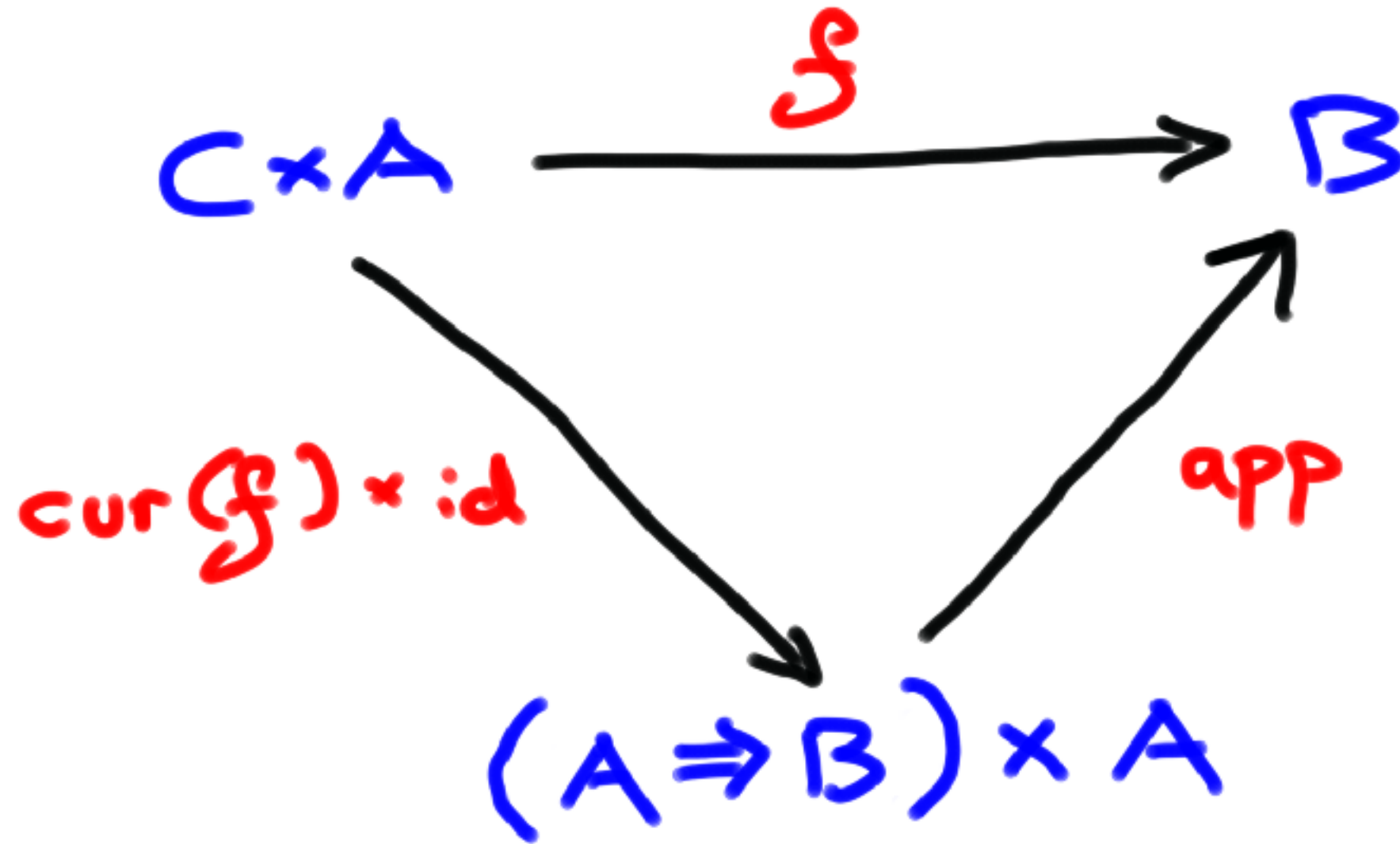
```
test = add one err
```

Exponentials

EXPONENTIALS



EXPONENTIALS



$$(\text{cur}(f) \times \text{id}) ; \text{app} = f$$

EXPONENTIALS

$$2 \Rightarrow 3 = 3^2$$

$$\begin{array}{l} 0 \mapsto 'a' \\ 1 \mapsto 'a' \end{array}$$

$$\begin{array}{l} 0 \mapsto 'b' \\ 1 \mapsto 'a' \end{array}$$

$$\begin{array}{l} 0 \mapsto 'c' \\ 1 \mapsto 'a' \end{array}$$

$$\begin{array}{l} 0 \mapsto 'a' \\ 1 \mapsto 'b' \end{array}$$

$$\begin{array}{l} 0 \mapsto 'b' \\ 1 \mapsto 'b' \end{array}$$

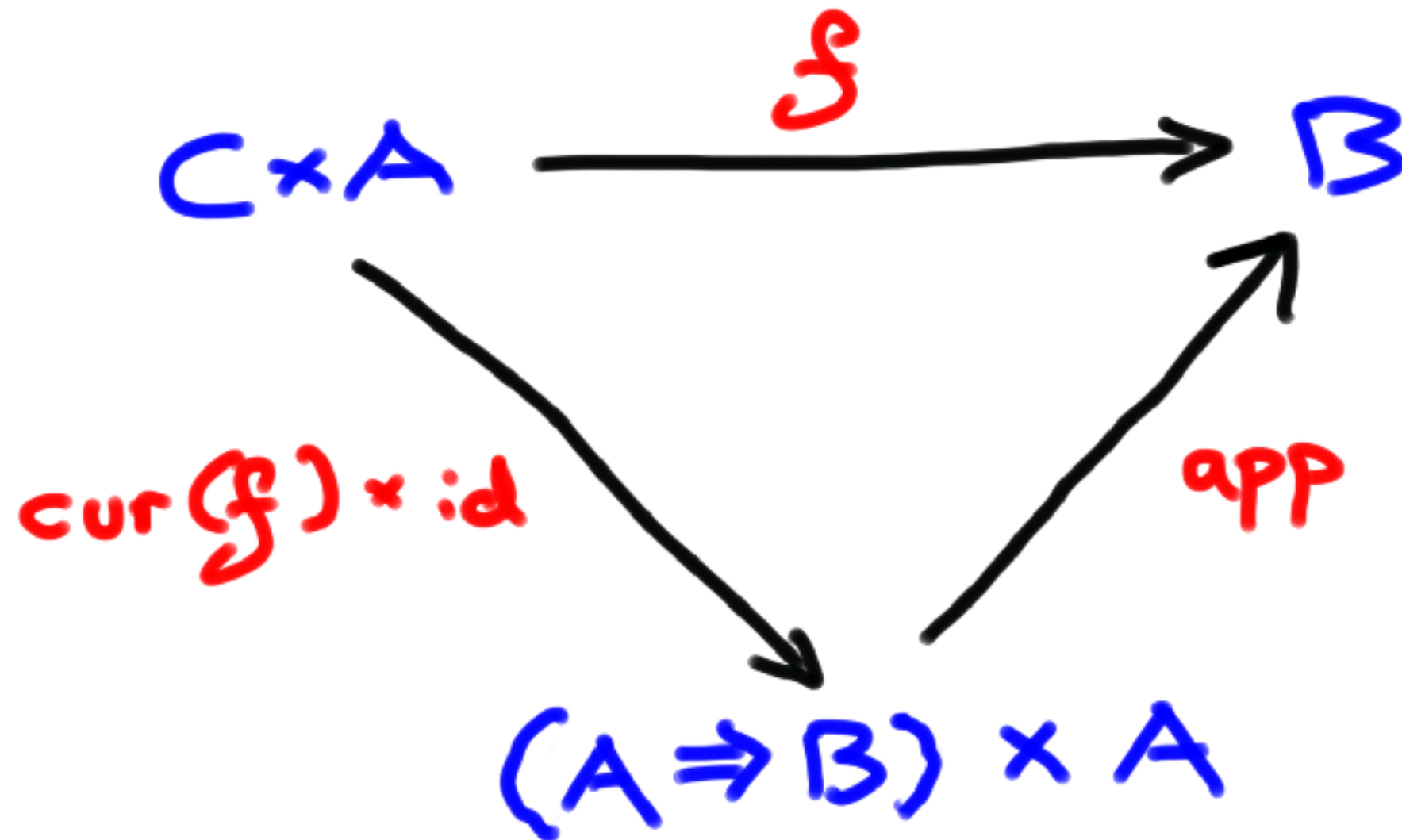
$$\begin{array}{l} 0 \mapsto 'c' \\ 1 \mapsto 'b' \end{array}$$

$$\begin{array}{l} 0 \mapsto 'a' \\ 1 \mapsto 'c' \end{array}$$

$$\begin{array}{l} 0 \mapsto 'b' \\ 1 \mapsto 'c' \end{array}$$

$$\begin{array}{l} 0 \mapsto 'c' \\ 1 \mapsto 'c' \end{array}$$

EXPONENTIALS



$$\mathcal{C}(C, A \Rightarrow B) \cong \mathcal{C}(C \times A, B)$$

Exponentials in Java

```
public class Test {  
    public Function<Integer,Integer>  
        add (Integer n) {  
        return x -> x + n;  
        }  
    public Function<Integer,Integer> incr =  
        add(1);  
    public Integer three = incr.apply(2);  
}
```

Exponentials in Haskell

```
add :: Int -> (Int -> Int)
add n = \x -> n + x
```

```
incr :: Int -> Int
incr = add 1
```

```
three :: Int
three = incr 2
```

Exponentials in Haskell

```
add :: Int -> Int -> Int  
add n x = n + x
```

```
incr :: Int -> Int  
incr = add 1
```

```
three :: Int  
three = incr 2
```

Simplicity & Michelson

Simplicity - Types

$$\begin{array}{c} \overline{\text{iden} : A \vdash A} \\[10pt] \overline{\text{unit} : A \vdash \mathbb{1}} \\[10pt] \frac{t : A \vdash B}{\text{injl } t : A \vdash B + C} \qquad \frac{t : A \vdash C}{\text{injrl } t : A \vdash B + C} \\[10pt] \frac{s : A \times C \vdash D \quad t : B \times C \vdash D}{\text{case } st : (A + B) \times C \vdash D} \qquad \frac{s : A \vdash B \quad t : A \vdash C}{\text{pair } st : A \vdash B \times C} \\[10pt] \frac{t : A \vdash C}{\text{take } t : A \times B \vdash C} \qquad \frac{t : B \vdash C}{\text{drop } t : A \times B \vdash C} \end{array}$$

Figure 1: Typing rules for the terms of core Simplicity.

Simplicity - Semantics

$$\llbracket \text{iden} \rrbracket(a) := a$$

$$\llbracket \text{comp } s \ t \rrbracket(a) := \llbracket t \rrbracket(\llbracket s \rrbracket(a))$$

$$\llbracket \text{unit} \rrbracket(a) := \langle \rangle$$

$$\llbracket \text{injl } t \rrbracket(a) := \sigma^{\mathbf{L}}(\llbracket t \rrbracket(a))$$

$$\llbracket \text{injrl } t \rrbracket(a) := \sigma^{\mathbf{R}}(\llbracket t \rrbracket(a))$$

$$\llbracket \text{case } s \ t \rrbracket \langle \sigma^{\mathbf{L}}(a), c \rangle := \llbracket s \rrbracket \langle a, c \rangle$$

$$\llbracket \text{case } s \ t \rrbracket \langle \sigma^{\mathbf{R}}(b), c \rangle := \llbracket t \rrbracket \langle b, c \rangle$$

$$\llbracket \text{pair } s \ t \rrbracket(a) := \langle \llbracket s \rrbracket(a), \llbracket t \rrbracket(a) \rangle$$

$$\llbracket \text{take } t \rrbracket \langle a, b \rangle := \llbracket t \rrbracket(a)$$

$$\llbracket \text{drop } t \rrbracket \langle a, b \rangle := \llbracket t \rrbracket(b)$$

Michelson - Sums

Operations on unions

- `LEFT 'b`: Pack a value in a union (left case). $:: 'a : 'S \rightarrow \text{or } 'a 'b : 'S$

 $> \text{LEFT} ; C / v :: S \Rightarrow C / (\text{Left } v) :: S$
- `RIGHT 'a`: Pack a value in a union (right case). $:: 'b : 'S \rightarrow \text{or } 'a 'b : 'S$

 $> \text{RIGHT} ; C / v :: S \Rightarrow C / (\text{Right } v) :: S$
- `IF_LEFT bt bf`: Inspect an optional value. $:: \text{or } 'a 'b : 'S \rightarrow 'c : 'S$
 $\text{iff } bt :: ['a : 'S \rightarrow 'c : 'S]$
 $bf :: ['b : 'S \rightarrow 'c : 'S]$

 $> \text{IF_LEFT} ; C / (\text{Left } a) : S \Rightarrow bt ; C / a : S$
 $> \text{IF_LEFT} ; C / (\text{Right } b) : S \Rightarrow bf ; C / b : S$

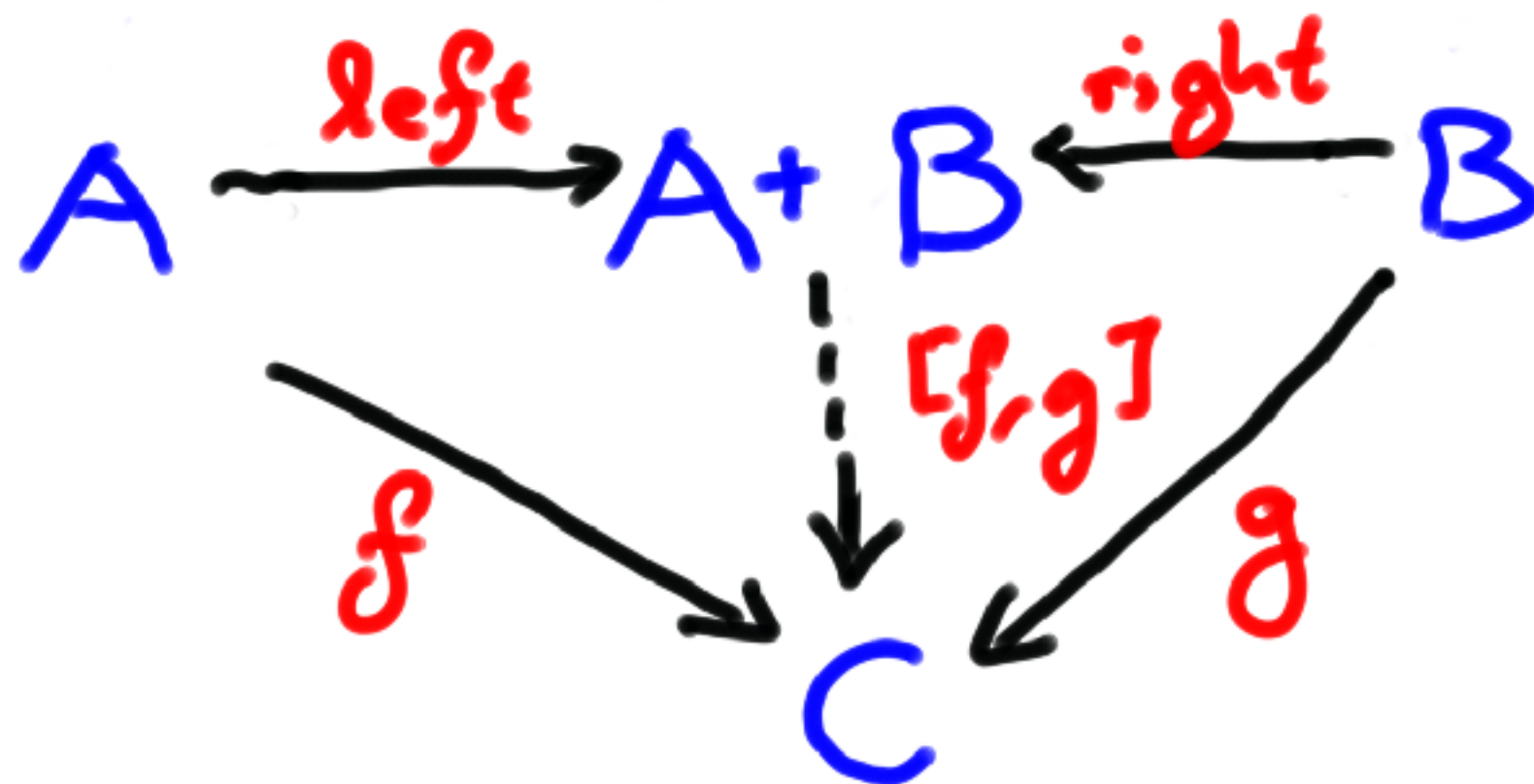
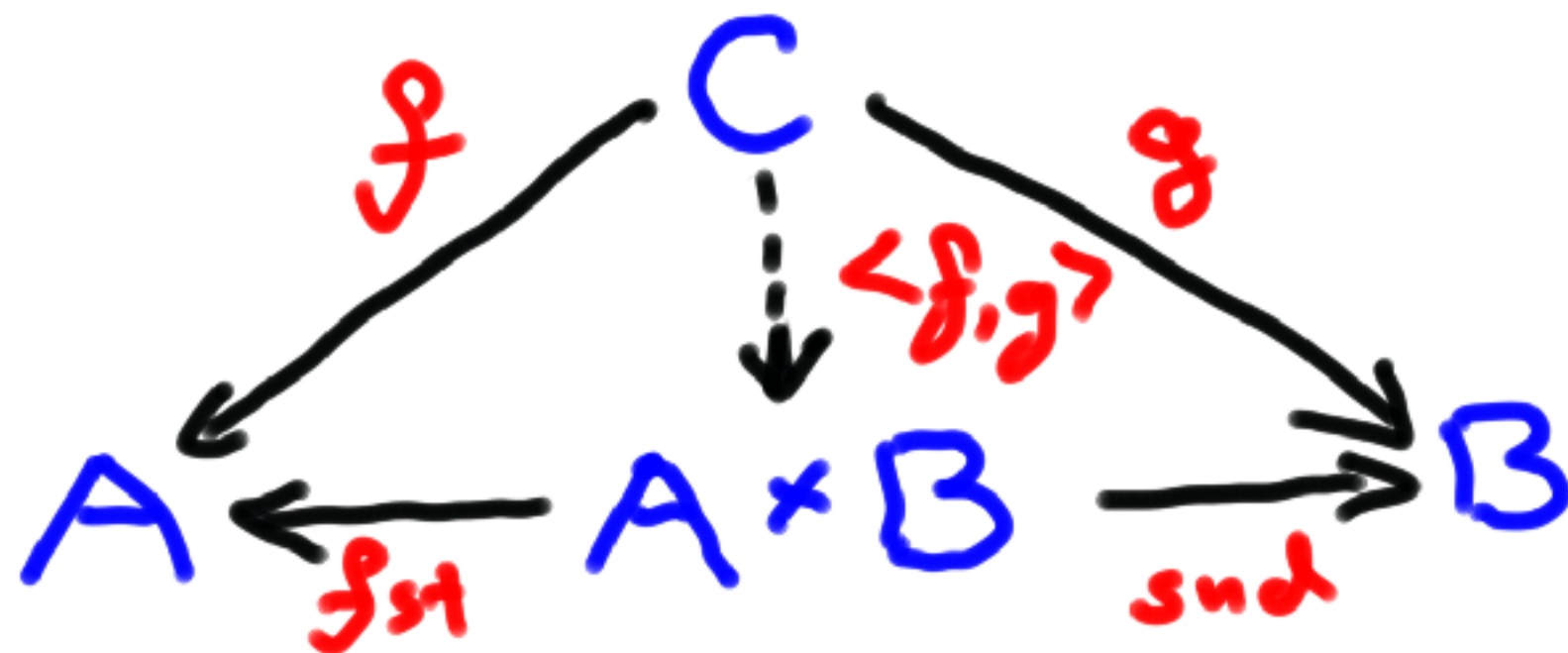
Plutus

IOHK is hiring 6
programming language
engineers

<https://iohk.io/careers/#op-235152-functional-compiler-engineer->

Conclusions

DUALS



ISOMORPHISMS

$$\mathcal{C}(C, A \times B) \cong \mathcal{C}(C, A) \times \mathcal{C}(C, B)$$

$$\mathcal{C}(A + B, C) \cong \mathcal{C}(A, C) \times \mathcal{C}(B, C)$$

$$\mathcal{C}(C, A \Rightarrow B) \cong \mathcal{C}(C \times A, B)$$

HIGH SCHOOL

$$(A \times B)^C = A^C \times B^C$$

$$C^{(A+B)} = C^A \times C^B$$

$$(B^A)^C = B^{C \times A}$$

Further Reading

- Saunders MacLane, *Categories for the Working Mathematician*
- Benjamin Pierce, *Basic Category Theory for Computer Scientists*
- Bartosz Milewski, *Programming Cafe* (blog)